

IBM Bob Workshop for the Java World — Participant Guide

Welcome

This workshop will help you get to know **IBM Bob** — an AI tool that helps Java developers understand unfamiliar legacy code, plan the migration from Java 8 to Java 21, and execute it in a controlled way.

What you will learn

- Analyze an unfamiliar Java 8 (legacy-style) application
- Ask IBM Bob for Java 21 modernization recommendations and rationale (what to change, why, and which Java 21 features to use)
- Make code changes in a controlled way
- Understand multi-layer applications (Swing UI + Service + Repository + Domain)
- Generate documentation, flow diagrams, and build configurations (Maven `pom.xml`, unit tests)
- Challenge IBM Bob's answers and verify them

Workshop duration

About **3 hours** (the practical part, after an opening session and demo).

Part One: Preparation

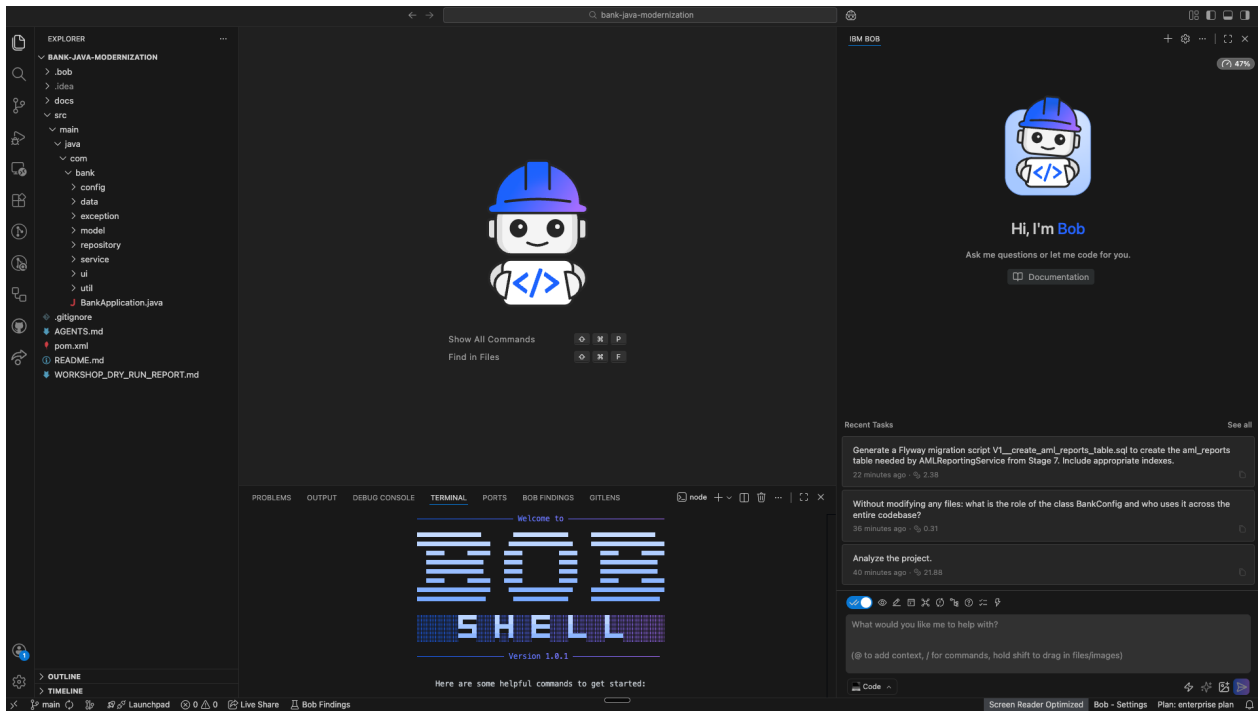
Pre-workshop checklist

- **IBM Bob IDE** installed and running on your machine
- **JDK 8** and **JDK 21** installed (you will need both: JDK 8 to run the legacy version, JDK 21 after modernization); verify with `java -version`
- **Maven 3.6+** installed (`mvn -v`)
- **Git client** installed
- Internet access available
- Prepare an empty folder (`C:\JavaBob\` on Windows, or `~/JavaBob/` on Mac)

Getting to know the IBM Bob IDE

Main windows:

- **Explorer** — file browser (left)
- **Editor** — code editor (center)
- **Chat** — chat with IBM Bob
- **Terminal** — command line



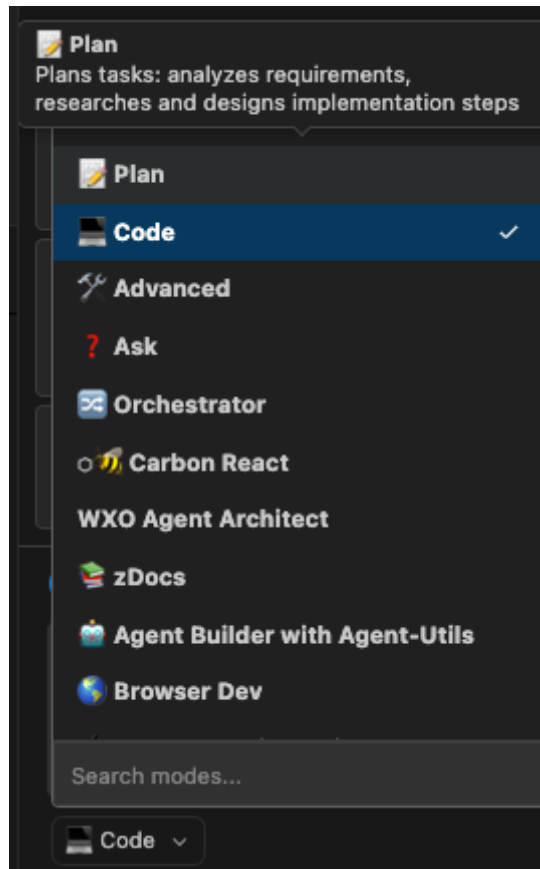
IBM Bob IDE overview: Explorer on the left, Editor in the center, Chat on the right, Terminal at the bottom

Modes:

IBM Bob works in two modes:

Mode	Behavior	When to use
PLAN / ASK	Advises, explains, recommends — does not change files	Stages 1–5
CODE	Changes files, actually runs commands	Stage 6 onward

The mode is shown at the bottom corner of the IDE. It is important to check it before every stage.



Active mode indicator (PLAN/ASK or CODE) at the bottom corner of the IBM Bob chat window

Part Two: Hands-on Work

Important: stages 1–7 are done **in a single shared chat** with IBM Bob. Do not open a new chat between these stages — IBM Bob remembers context within one conversation, and many prompts in stages 4–7 build on what was discussed earlier. A new chat is opened explicitly only starting from stage 8 (where the stage itself says “Open a new chat”).

Stage 1 — Preliminary: download the project

Goal

Get to know IBM Bob’s working environment through a practical task — downloading the **bank-java-modernization** project.

Full process

a. Paste into the IBM Bob chat:

Download the project from <https://github.com/igor-olikh/bank-java-modernization> and load it into IBM Bob.

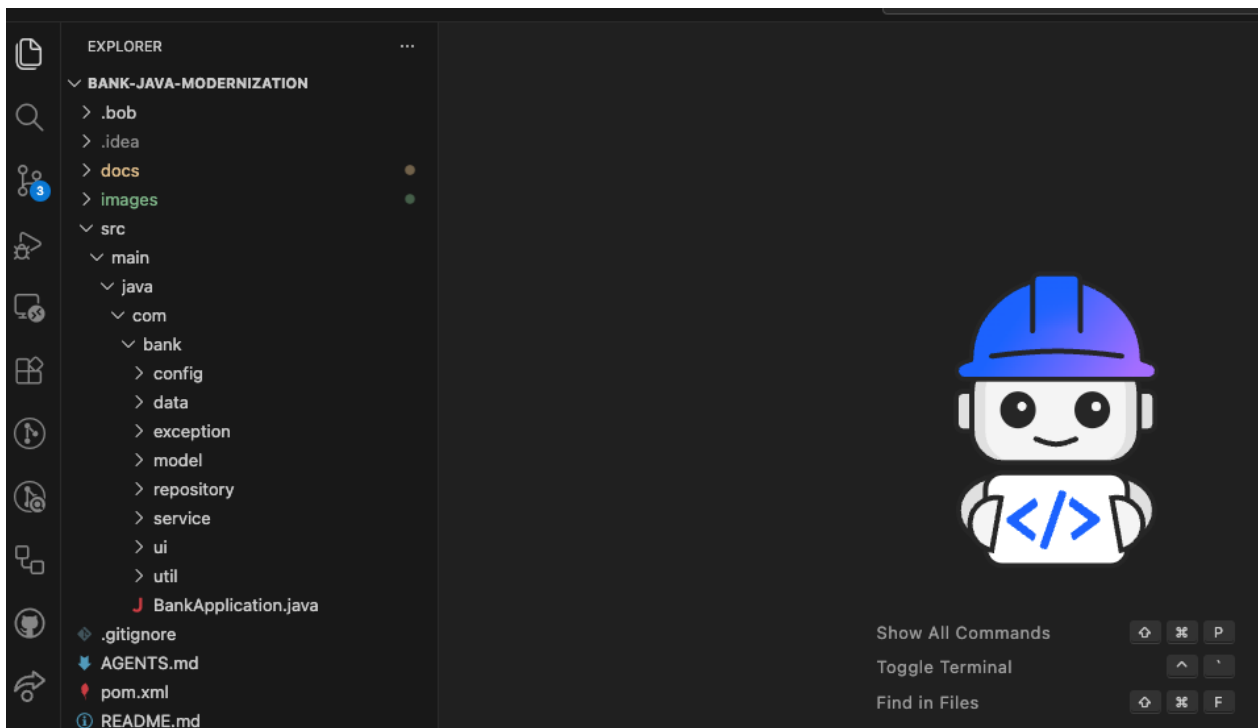
b. IBM Bob will present a plan and will likely recommend using `git client`.

c. Direct IBM Bob to do a direct ZIP download into the folder you prepared:

I prefer a direct download of a ZIP file from the site into the folder `C:\JavaBob\` (or `~/JavaBob/` if you are on Mac).

d. Open the project in the IDE:

1. Menu: **File** → **Open Folder**
2. Select the `bank-java-modernization` folder you created
3. Confirm the opening



Explorer in IBM Bob IDE with the bank-java-modernization project open

Expected results

- A local copy of the project on your machine
- Familiarity with the main IDE windows
- An initial understanding that IBM Bob automates tasks around the code, not just the code itself

Note: to download the project, IBM Bob will briefly switch to **CODE** mode (because it needs to write files to disk) — confirm the switch. This is not the focus of this stage; we will return to it in detail in stage 6. Before moving on to stage 2, make sure you returned to **PLAN/ASK** mode.

Tip: if IBM Bob does nothing — it is most likely waiting for an answer from you. Check the chat.

Bonus (optional): after opening the project, ask IBM Bob:

Build the project with Maven and confirm that the build succeeds.

IBM Bob will switch to **CODE** mode, run `mvn clean package`, and show the result. This immediately confirms that JDK and Maven are configured correctly. After that, do not forget to return to **PLAN/ASK** mode.

Stage 2 — Analysis

Goal

Understand what the application does, which modules exist, and how everything connects — without opening a single Java file.

Stage 2.1 — General analysis

Paste into IBM Bob:

Analyze the project.

If IBM Bob asks a clarifying question (for example, “What aspects should I focus on?”) with suggested options — pick the most general option (“All aspects”, “General overview”), or write in your own words: `All aspects – give me a comprehensive overview`. This applies to other stages too — IBM Bob periodically asks clarifying questions before answering.

IBM Bob will produce an analysis document explaining the application.

Then go deeper with this prompt:

Explain the application at a high level, including business goals, core functions, interfaces, and the system architecture.

What you should get

- An executive summary of the application
- Business goals (customer, account, transfer management)
- Core functions
- Architecture (Java 8, Swing, Maven, in-memory repositories, MVC pattern)
- A map of the main services and layers (`service/` , `repository/` , `model/` , `ui/`)

Practical tip: it's recommended to ask IBM Bob to save the analysis to a file so you can return to it later:

Save the analysis to a file `ANALYSIS.md` .

Open the created file — click the open preview icon.

Stage 2.2 — Analysis of a specific operation (money transfer between accounts)

Paste into IBM Bob:

Explain the money transfer operation, including: which classes are involved, dependencies, data flow, exceptions, UI integration, and persistence.

What you should get

IBM Bob will provide a detailed analysis of the **money transfer** operation, which will include (among other things):

- Identification of the operation — a transfer between two accounts of the same or different customers
- Main method — `TransactionService.transfer(fromAccountId, toAccountId, amount, description)`
- Related UI screen — `TransactionsPanel`
- List of called services (`AccountService.getAccount` , `AccountService.saveAccount` , `TransactionRepository.save`)
- Dependencies and interfaces — models (`Account` , `Transaction`), repositories, exceptions (`InsufficientFundsException` , `AccountFrozenException`)
- Flow description — from a UI click through the service to the repository and back

IBM Bob's response may be richer or slightly different on each run. This is natural for working with an AI tool. The main thing is that the answer covers the main topics.

Stage 3 — Application change request

Goal

Simulate a situation where a new business request reaches a Java developer, and see how IBM Bob translates it into recommendations.

Business context

A new regulatory requirement — report every money transfer above **9,000 NIS**, for anti-money-laundering (**AML**) purposes.

Prompt

Paste into IBM Bob:

Without modifying any files: what would need to change in order to report into an external file or external table every money transfer between accounts in an amount above 9,000 NIS? All relevant details of the operation must be reported (such as date, time, amount, source, destination, etc.). Just describe the recommended changes — do not edit or create any files.

What you should get

IBM Bob will provide:

- Identification of the correct class/method to change:
`TransactionService.transfer(...)` (file `src/main/java/com/bank/service/TransactionService.java`)
- A description of the required change
- A list of fields to report (date, time, amount, accounts, user, terminal, etc.)

Note: IBM Bob sometimes gives more than you asked for — possibly even a comparison between options, possibly even code. That's fine — it's thorough.

Stage 4 — Going deeper (justifying the recommendation)

Goal

See that IBM Bob can justify its recommendations based on the code. This turns it into a discussion partner, not just a tool that returns answers.

Context

Like `TransactionService.transfer()`, the `TransactionService.deposit()` and `TransactionService.withdraw()` methods also work with accounts. So why the change in one method and not the others? IBM Bob should explain that this is because `deposit()` and `withdraw()` operate on a single account (credit/debit), while `transfer()` handles a transfer between two accounts.

Prompt

Paste into IBM Bob:

Without modifying any files: explain why the change is needed in method `TransactionService.transfer()` and not in `TransactionService.deposit()` or `TransactionService.withdraw()`.

What you should get

<code>transfer()</code>	<code>deposit()</code> / <code>withdraw()</code>
Handles a transfer between two accounts	Handle a single account (credit or debit)
Parameters: <code>fromAccountId</code> and <code>toAccountId</code> (source + destination)	Parameter: <code>accountId</code> (single account)
Matches the AML business requirement	Match deposits/withdrawals (do not require AML reporting)

IBM Bob will show **method signatures** and their **bodies** as proof (`public Transaction transfer(...)`, `public Transaction deposit(...)`, `public Transaction withdraw(...)`).

Stage 5 — Decision-support information

Goal

Improve the Java developer's judgment by comparing implementation options before coding.

Prompt

Paste into IBM Bob:

Without modifying any files: what is the preferred storage for AML reports — a relational database (JDBC/JPA), a NoSQL store (MongoDB), a structured file (CSV/JSON), or a message queue (Kafka)?

What you should get

- A comparison between using a **relational DB (JDBC/JPA)**, **NoSQL (MongoDB)**, a **structured file (CSV/JSON)**, and a **message queue (Kafka)**
- Pros and cons of each option
- IBM Bob’s reasoned recommendation — work with a **relational DB (e.g. PostgreSQL via JPA/Hibernate)**, since AML requirements include audit, ACID transactions, and SQL queries for checks

Tip: if IBM Bob already answered this in stage 3, you can skip it or just ask a specific question (e.g. “What is the impact on performance?”).

Stage 6 — Modernization Java 8 → Java 21 (/java-modernization mode)

Goal

Demonstrate IBM Bob’s specialized mode for Java modernization — `/java-modernization`. In this mode, IBM Bob automatically analyzes, plans, and executes the migration from Java 8 to Java 21. **You don’t need to write prompts** — you only review proposals and confirm actions.

Context

In the regular PLAN/ASK and CODE modes, we manually asked IBM Bob questions and gave instructions. But for Java modernization, IBM Bob has a separate, pre-trained mode — `/java-modernization`. This is the right way to modernize an enterprise application, and that’s exactly what we’ll show.

Stage 6.1 — Activating the mode

In the IBM Bob chat:

1. Type `/` — the **Modes** dropdown will appear
2. Select `/java-modernization` (description: Modernize Java applications)
3. The active-mode indicator at the bottom-left of the chat will switch to — **Java Modernization**

After activation, IBM Bob is ready for modernization. Long prompts are no longer needed — the mode already knows what to do.

Stage 6.2 — Running the modernization (two phases)

IBM Bob naturally splits the modernization into **planning** and **execution**. First it shows a plan and asks for an explicit “go ahead” — this is your additional control before the bulk changes.

Stage 6.2 (a) — Request the plan

Paste:

Modernize this project from Java 8 to Java 21.

IBM Bob will:

- Analyze the codebase
- Create a migration plan file (usually `JAVA_21_MIGRATION_PLAN.md`)
- Possibly update `pom.xml` immediately (source/target → 21)
- **Stop** and show the plan: what to change, which Java 21 features to apply
- Wait for your “apply or not” decision

Stage 6.2 (b) — Trigger execution

When you’ve seen the plan and want to apply it, paste:

Now execute the migration plan you produced. Apply all phases: convert models to Records, make BankException sealed, apply Virtual Threads, Pattern Matching, Text Blocks, and modernize repositories. Show me each diff before applying, and ask for approval on every file change.

IBM Bob will start **applying changes**, step by step, asking confirmation on each one.

Stage 6.3 — Approvals (your main role)

This is the key moment: **your job is not to type, but to review and approve**.

IBM Bob will periodically pause and ask for confirmation. Typical dialogs:

- “Change `pom.xml` (source/target 1.8 → 21)?” → **APPROVE**
- “Convert `Transaction.java` to a record? (60 lines → 8 lines)” → review the diff → **APPROVE**
- “Apply Virtual Threads in `TransactionService`?” → **APPROVE**
- “Replace `if/else if` with a pattern-matching `switch` in `TransactionMenu`?” → **APPROVE**

Tip: if you want to see details before each confirmation, ask IBM Bob:

Always show me the exact diff before applying any change.

IBM Bob may offer choices with trade-offs. On some files (for example, `Transaction.java`, which has a mutable `status` field) IBM Bob will see several reasonable paths and offer a choice — for example:

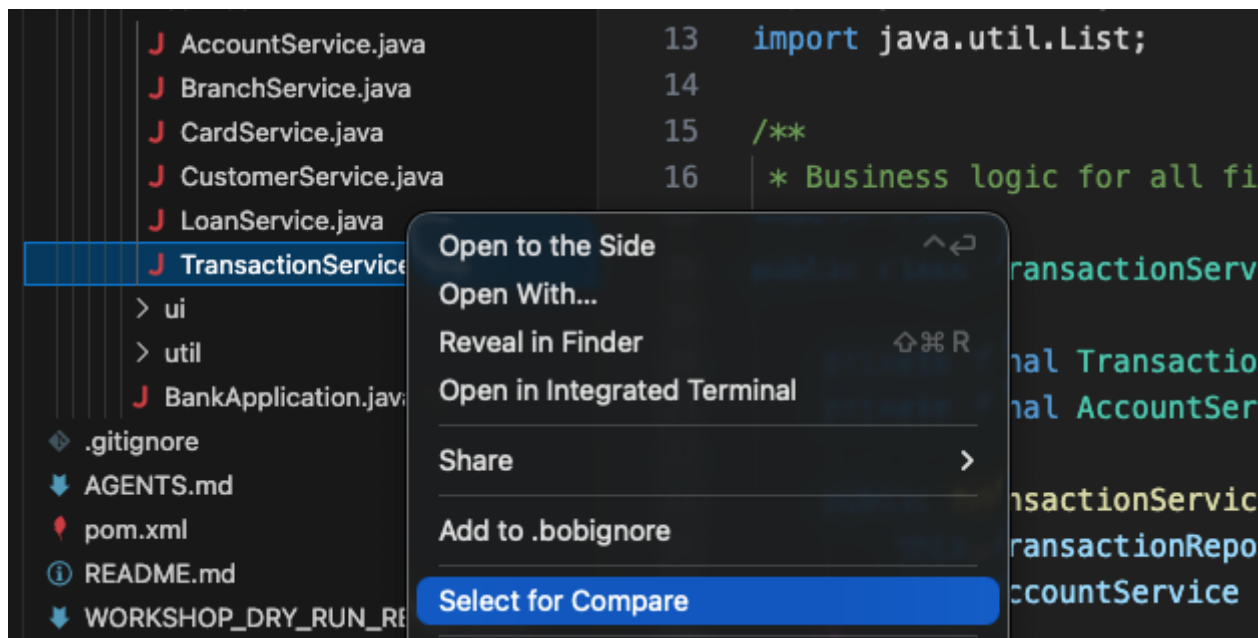
- **Option A:** keep as a class, only modernize syntax (`var`, `switch`) — safer, no breaking changes
- **Option B:** convert to a record + add an immutable wrapper — more Java 21 idioms, but requires changes in dependent places

IBM Bob usually gives its own recommendation (often the “safe” option). Pick whichever suits you — both options are valid. If you want to maximally show off Java 21 features — pick the more “advanced” option (record / sealed / pattern matching). If you want minimal risk — pick the recommended one.

Stage 6.4 — Before/after comparison

After each block of changes, compare the original to the modernized version:

- In the **Explorer**, right-click the new version of the file → **Select for Compare**
- Then right-click the original → **Compare with Selected**



File comparison in IBM Bob: the Select for Compare / Compare with Selected context menu, or the open diff view

The most striking moment is `Transaction.java` : ~60 lines of boilerplate become a single Record line.

What IBM Bob will produce (typical result)

- **Updated pom.xml** (`<source>21</source>` , `<target>21</target>` , current plugins added)
- **Models as Records** — `Transaction` , `Address` , simple DTOs
- **Sealed hierarchies** for transaction types / account statuses
- **Pattern-matching switch** where there used to be a long `if/else if`
- **Virtual Threads** in `TransactionService` for parallel processing
- **Text Blocks** in `DataSeeder` and other places with long strings
- **Modern HttpClient** (if it finds `HttpURLConnection`)
- **A MODERNIZATION_NOTES.md document** explaining each change and the rationale for the chosen Java 21 features

Stage 6.5 — Build and verify

After the modernization is done, ask IBM Bob:

Build the modernized project with Maven using JDK 21 and run it to verify everything works.

IBM Bob will run `mvn clean package` on JDK 21 and confirm that:

- The project builds without errors
- Unit tests pass (if any exist)
- The application launches

Why this matters

- **IBM Bob's specialized modes** — modernization is launched with one command, not a series of manual prompts. IBM Bob has ready-made modes for specific enterprise scenarios.
- **The visible “60 lines → 1 line” effect** — Records make the code drastically shorter and clearer.
- **Change safety** — Sealed Types + Pattern Matching force the compiler to catch missed cases. New transaction types won't silently fall through to `default` .
- **Real performance impact** — Virtual Threads let the same server serve 10–100× more clients.
- **Controllability** — despite the automatic mode, every change passes through developer approval.

Before moving on to stage 7, make sure that:

- All changes are saved and approved
- The build succeeds on JDK 21
- The application launches

- IBM Bob has been switched back to **PLAN/ASK** mode (or the default mode)
-

Stage 7 — Implementing the AML feature on the modernized code

Goal

Implement the very AML feature we discussed in stages 3–5 — now on modern Java 21. Show how the new language features (Records, Virtual Threads) simplify implementing new business functionality.

Context

In stage 3, IBM Bob identified that the AML check should be added to `TransactionService.transfer()`. In stage 5, IBM Bob recommended storing reports in a relational DB. After stage 6, the codebase is already on Java 21 — it's the right time to implement AML.

Before starting, make sure IBM Bob is back in regular **PLAN/ASK** mode (not `/java-modernization`).

Stage 7.1 — Implementation plan

Paste into IBM Bob:

Now that the project is on Java 21, implement the AML reporting feature we discussed earlier: report every transfer above 9,000 NIS to an AML store. Use Java 21 features where appropriate (a Record for the report DTO, Virtual Threads for asynchronous writing).

What you should get (PLAN mode)

- A plan to create a new `AmlReportService`
- A new Record `AmlReport` (with fields date, time, amount, fromAccount, toAccount, user, etc.)
- Changes in `TransactionService.transfer()` — checking the 9,000 NIS threshold and calling the AML service
- An approach to storage (in-memory for the demo, JDBC/JPA-ready interface for production)
- An approach to asynchronous writing via Virtual Threads, so as not to block the main transfer

Stage 7.2 — Applying the changes

Once the plan is approved:

Apply the plan. Show me each diff before approving.

IBM Bob will request a switch to CODE, show the diffs, and apply the changes upon confirmation.

Stage 7.3 — Optional: comparison with the “old” style

If time permits, ask IBM Bob to show how the same feature would have looked in Java 8 style:

Show me how this AML feature would have looked if implemented in the original Java 8 style, in a separate file named `AmlReportServiceJava8Style.java`. Highlight the differences.

This will let you visually appreciate **how much shorter and safer Java 21 makes the implementation**.

What IBM Bob will produce

- **AmlReport** — a Record with typed fields (no boilerplate)
- **AmlReportService** — a service with asynchronous writing via Virtual Threads
- **Modified TransactionService.transfer()** — threshold check added and `AmlReportService` is called
- **A unit test** for the AML logic
- **An AML_IMPLEMENTATION.md file** describing the changes and the rationale

Why this matters

- **Less boilerplate** — a Record removes the need to write getters, `equals`, `hashCode` by hand
- **Doesn't block the main thread** — asynchronous writing via Virtual Threads doesn't slow down the transfer itself
- **Production-ready** — the service structure and storage interface make it easy to switch from in-memory to a real DB
- **Traceability** — every change is documented in `AML_IMPLEMENTATION.md`

Before moving on to stage 8, make sure that:

- The build succeeds
 - The unit test for the AML logic passes
 - IBM Bob is back in **PLAN/ASK** mode
-

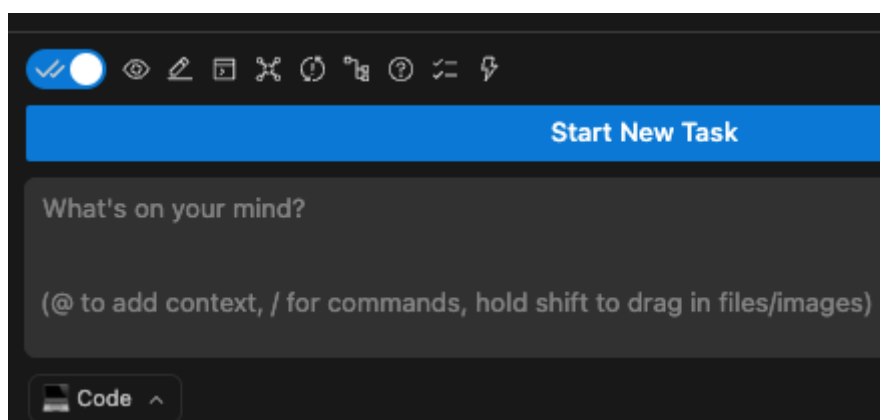
Stage 8 — Understanding undocumented components (BankConfig)

Goal

See that IBM Bob can give **whole-codebase understanding** — find all places where a class or constant is used, even when there is no documentation in the code about its consumers.

Prompt

Open a new chat (+).



The “+” button to start a new chat (new task) at the top of the IBM Bob chat window

Paste into IBM Bob:

Without modifying any files: what is the role of the class `BankConfig` and who uses it across the entire codebase?

What you should get

IBM Bob will explain:

- Role: a centralized brand configuration — bank name, slogan, support phone, SWIFT code, UI colors
- How to rename the bank: change a **single constant** `BANK_NAME` and the entire UI will reflect the change
- A list of all consumers: `MainWindow.java`, panels in `ui/panels/`, `DataSeeder.java`, etc. — with concrete lines

Why this matters

In real banking systems, you often have a situation where a single class with constants or utilities is used by dozens of other classes. Searching for all usages by hand is a slow process. IBM Bob does it in seconds and gives a whole-codebase view of dependencies.

Additional exercise (recommended): challenge IBM Bob

IBM Bob sometimes says things it has not proven. It's worth challenging it:

Show me the specific code that proves your claims. Actually search in the files.

What to expect

- IBM Bob will correct itself if it was wrong
- It may discover additional things in the code (e.g., unused variables)

Important lesson: always challenge AI when something sounds exaggerated. The tool is more powerful the better you know how to demand evidence.

Stage 9 — Infrastructure around the application (Maven, Docker, CI/CD)

Goal

Show IBM Bob's value to the Java developer also in areas adjacent to Java but not part of business logic — build (Maven), containerization (Docker), CI/CD (GitHub Actions), DB migrations.

Context

In stage 2, IBM Bob explained how the application handles the customer topic. After stage 7 we have an AML feature, and AML data needs to be stored in a real DB. Now we'll ask practical questions about supporting infrastructure around the application.

Switch back to **PLAN** mode.

Stage 9.1 — Containerization (Docker)

Paste into IBM Bob:

Create a `Dockerfile` for this Java 21 application based on `eclipse-temurin:21-jdk-alpine`. The image should run `bank-system.jar` and expose any necessary ports.

What you should get:

- A ready `Dockerfile` with a multi-stage build (build stage + runtime stage)
- A `.dockerignore` file
- An explanation of each line

Stage 9.2 — CI/CD (GitHub Actions)

Paste into IBM Bob:

Just create the `.github/workflows/build.yml` file locally on disk that builds the project with JDK 21 and runs the tests on every push. Do NOT push to GitHub or interact with any GitHub authentication. Just create the file content locally.

What you should get:

- A `.github/workflows/build.yml` file with standard steps: `actions/checkout`, `actions/setup-java`, `mvn verify`
- Triggers on `push` and `pull_request`

Stage 9.3 — DB migration (Flyway) for the AML table

Paste into IBM Bob:

Generate a Flyway migration script `V1__create_aml_reports_table.sql` to create the `aml_reports` table needed by `AmlReportService` from Stage 7. Include appropriate indexes.

What you should get:

- A SQL script `CREATE TABLE aml_reports` with typed columns
- Indexes on date and amount (for frequent queries)
- An explanation of the schema and the indexing strategy

Why this matters

IBM Bob is fluent across the whole stack around a Java application: build (Maven), containers (Docker), CI (GitHub Actions / Jenkins), DB migrations, logging. The developer doesn't have to switch between several specialists — IBM Bob covers all the surrounding plumbing.

Stage 10 — Additional questions

Some of these questions were shown in today's opening demo, but it is also useful to go through them on your own.

Pick questions based on time and interest:

Question 10a — Account opening checks

Without modifying any files: which conditions are checked before opening an account, and where is this expressed in the code?

What this demonstrates: IBM Bob can extract business rules from the code.

Question 10b — Changing the account number

Without modifying any files: what is the implication of changing the account number from 8 digits to 10 digits, and what needs to be changed?

What this demonstrates: IBM Bob can trace a data-structure change throughout the code.

Question 10c — Writing a JUnit test

Write a JUnit 5 test class for the 'open account' process in `AccountService.openAccount()`. Include both positive and negative test cases.

Important: first inspect the current code (`Account` , `Customer` , `Address`) to use the correct constructor signatures — the project was recently modernized to Java 21, so models like `Address` may now be Records with different field counts. Make sure your test code is consistent with the current state of the project, then compile and run `mvn test` to verify.

What this demonstrates: IBM Bob doesn't just analyze — it also writes modern Java test code.

Question 10d — Business flow description

Without modifying any files: describe the business steps of 'money transfer' in the system, including business rules and controls. Provide a link to the code lines.

What this demonstrates: IBM Bob can translate code into a clear business description for managers.

Question 10e — Mermaid flow diagram

Create a flow diagram in Mermaid style for the money transfer process (showing UI → Service → Repository layers). Save it to a file `TRANSFER_FLOW.md`.

What this demonstrates: IBM Bob produces visual documentation.

Tip: to see rendered Mermaid diagrams in IBM Bob preview, install the **Markdown Preview Mermaid Support** extension.

Question 10f — Advanced challenge: T-PIN authentication

Without modifying any files: below is a new business requirement: add an additional identity verification before performing a money transfer in `TransactionService.transfer()`. The verification before the transfer is by a random one-time 6-digit code, the **Transaction PIN (T-PIN)**, which will be shown on the screen and the user is required to enter it. Where does the change need to be made, suggest how to implement the change, and present a proposed code change using Java 21 idioms (e.g. a `record` for the T-PIN challenge). Describe only — do not edit or create any files.

What this demonstrates: a full, complex business requirement — from analysis through an implementation proposal to code.

Summary

What you learned

- IBM Bob enables fast analysis of an unfamiliar application
- IBM Bob works across technologies (Java, SQL, Maven, Docker, CI/CD)
- IBM Bob operates in a controlled way — **PLAN**, approval, only then **CODE**
- IBM Bob can be wrong — therefore it's important to challenge it and verify
- IBM Bob also saves significant time on tasks that support the code (documentation, infrastructure configs, scripts)
- IBM Bob has specialized modes — for example, `/java-modernization` for migrating Java 8 → Java 21

Five key tips

1. Start in PLAN, switch to CODE only when you have to. Always start in **PLAN/ASK** to get recommendations. Only when you are confident, switch to **CODE** for execution.

2. Save analyses to files. Ask IBM Bob: “Save the analysis to file `XYZ.md`”. This lets you come back to them later.

3. Challenge unproven claims. If IBM Bob says something that looks unsupported: “Show me the code that proves this. If there is none — state that explicitly.”

4. Give files explicit names. In prompts: “Save this to file `XYZ.md`”, not just “Save this”. This prevents mistakes.

5. Use IBM Bob’s specialized modes. For modernizing Java applications, activate the `/java-modernization` mode — it already knows the best practices for migrating to Java 21. No long prompts needed.

Additional help

- **IBM Bob expert** — for tool operation
- **Java modernization expert** — for questions about migrating to Java 21
- **Java/Spring expert** — for technical Java questions

Good luck with the workshop!